`pyhrp`: Hierarchical Risk Parity with a Pluggable Risk Model and Polyhedral Constraints

Thomas Schmelzer*

November 13, 2026

Abstract

Hierarchical Risk Parity (HRP) fixes three choices at once: the covariance estimate, the cluster tree built from it, and the rule that splits the budget down that tree. This paper loosens HRP in two ways. First, we disconnect the risk model from the allocation scheme. The covariance becomes an input — a sample, shrunk or factor-model estimate — and every allocation rule runs on any tree, so HRP, its Schur-complement extension and a hierarchical $1/N$ become interchangeable rules over a shared tree. Second, HRP honours no constraint beyond long-only and fully invested. We use the critical line algorithm (CLA) to find the portfolio closest to HRP, measured in the risk model's own metric, that satisfies an arbitrary set of polyhedral constraints. No new solver is needed: setting the expected returns to Σw^{HRP} turns the nearest-portfolio problem into one point on the CLA path, and when the HRP portfolio is already feasible it comes back unchanged.

1 Introduction

Hierarchical Risk Parity (López de Prado, 2016) avoids the matrix inversion that makes mean-variance optimisation (Markowitz, 1952) unstable (Michaud, 1989). It clusters the assets into a binary tree and then splits a unit budget down that tree, giving each of a node's two children a share inversely proportional to its variance. The result is long-only, fully invested and needs no expected returns.

As published, HRP is one fused pipeline: prices go in, a sample covariance is estimated, a tree is built from the implied correlations, and the same covariance drives the split. This paper pulls that pipeline apart and puts it back together under constraints.

Part I: the risk model as an input. `pyhrp` separates the *risk model*, the covariance Σ, from the *allocation scheme*, the rule that splits the budget at each node. The allocators take Σ as an argument — an explicit matrix, or an operator that only multiplies, as for a factor model — and take the tree as an argument too. Shrinkage (Ledoit and Wolf, 2004), a factor model or any other estimate can therefore replace the sample covariance without touching the allocator, and the allocator can change without rebuilding the tree. Three schemes share this contract: HRP's inverse-variance split, the Schur-complement split of Cotton (2024), and a hierarchical $1/N$. The tree itself is returned, not just a weight vector, so the budget assigned to every cluster can be inspected.

Part II: HRP under polyhedral constraints. Real mandates impose bounds, sector limits and budget constraints, and HRP cannot honour them. Rather than modify the recursion, we take the HRP portfolio as a target and ask for the feasible portfolio nearest to it. Measured

*`https://github.com/tschm/pyhrp`

in the Σ-norm, that is the portfolio with the smallest tracking error to HRP. This is a quadratic program over a polyhedron, and the critical line algorithm (Markowitz, 1956; Schmelzer and Hastie, 2026) solves it exactly.

Sections 2–4 cover Part I: the construction, its implementation, and numerical results. Section 5 covers Part II.

2 Method

Let $R \in \mathbb{R}^{T \times N}$ be a matrix of simple returns for N assets over T periods, with sample covariance $\Sigma \in \mathbb{R}^{N \times N}$ and sample correlation $C \in \mathbb{R}^{N \times N}$.

2.1 From correlation to a tree

Correlations are turned into distances by the standard map

$$d_{ij} = \sqrt{\frac{1 - C_{ij}}{2}}, \tag{1}$$

clipped to $[0, 1]$ before the square root and with an exactly zero diagonal. Perfectly correlated assets sit at distance 0, uncorrelated assets at $1/\sqrt{2}$, perfectly anticorrelated assets at 1.

The condensed form of d is passed to SciPy's agglomerative clustering (Virtanen et al., 2020) under one of four linkage methods — `single`, `complete`, `average` or `ward` (Ward, 1963), the default. The resulting linkage matrix is converted into a binary tree of `Cluster` nodes, each leaf carrying the column index of one asset.

The correlation matrix is validated before use. Fewer than two assets, or any non-finite entry, raises. A non-finite diagonal entry is reported with the offending asset named, because the cause is almost always a constant price series, whose zero variance makes its correlations undefined.

The bisection variant. Setting `bisection=True` keeps only the *leaf order* that the chosen linkage induces and discards the tree shape. The ordered list of leaves is then split in half, recursively, until singletons remain. This is closer to the construction described in the original HRP paper, and it produces a balanced tree by definition: depth is $\lceil \log_2 N \rceil$ regardless of how chain-like the linkage was.

A bisection tree has no linkage matrix of its own, so one is rebuilt for plotting. The merge height assigned to an internal node is its *cluster diameter*,

$$h(v) = \max_{i,j \in \text{leaves}(v)} d_{ij}, \tag{2}$$

which is monotone along the tree — a parent's leaf set contains each child's, so its maximum can only grow — and monotonicity is exactly what SciPy requires of that column.

2.2 The recursive split

Every node holds a `Portfolio`: a mapping from asset name to weight. A leaf holds its own asset at weight 1. An internal node combines its children.

For an internal node v with children ℓ and r, let w_ℓ and w_r denote the child portfolios — each already a unit-sum allocation over its own leaf set — and let Σ_ℓ, Σ_r be the corresponding diagonal blocks of Σ. Define the child variances

$$v_\ell = w_\ell^\top \Sigma_\ell w_\ell, \qquad v_r = w_r^\top \Sigma_r w_r. \tag{3}$$

The budget of v is split

$$\alpha_\ell = \frac{v_r}{v_\ell + v_r}, \qquad \alpha_r = 1 - \alpha_\ell = \frac{v_\ell}{v_\ell + v_r}, \tag{4}$$

so that $\alpha_\ell v_\ell = \alpha_r v_r$: the two children contribute equally to the parent, up to the cross term, which this rule ignores. If both variances vanish — riskless sub-portfolios — the split is $\alpha_\ell = \alpha_r = 1/2$ rather than undefined. The node's portfolio is

$$w_v = \alpha_\ell \, w_\ell \oplus \alpha_r \, w_r, \tag{5}$$

where $\oplus$ concatenates over disjoint asset sets. Since $\alpha_\ell + \alpha_r = 1$ and both are non-negative, the invariant that a node's weights are non-negative and sum to one propagates from the leaves to the root.

Algorithm 1. Allocation on a cluster tree

Input: root ρ, covariance Σ, per-node variance rule ν.

1: $\mathcal{O} \leftarrow$ nodes of ρ in pre-order, collected with an explicit stack so that every parent precedes its children.

2: For each $v \in \text{reverse}(\mathcal{O})$:

- if v is a leaf, set $w_v \leftarrow \{\, \text{asset}(v) : 1 \,\}$;
- otherwise compute $(v_\ell, v_r) \leftarrow \nu(\ell, r, \Sigma)$ — Eq. (3) for HRP, Eq. (8) for Schur — and set $w_v \leftarrow \alpha_\ell w_\ell \oplus \alpha_r w_r$ by Eq. (4).

3: Return ρ.

Algorithm 1 is the whole allocator. Reversing a pre-order walk guarantees that both children carry a portfolio before their parent needs one, which is what makes a single bottom-up pass sufficient.

2.3 Schur complementary allocation

Equation (3) uses only the diagonal blocks of Σ. The cross-block covariance between the two children is discarded — at every node, at every level. Schur Complementary Allocation (Cotton, 2024) restores part of it.

Write the covariance restricted to the union of the two children's assets in block form,

$$\Sigma_v = \begin{pmatrix} A & B \\ B^\top & D \end{pmatrix}, \tag{6}$$

with A the left block and D the right. The augmented blocks are

$$A_\gamma = A - \gamma \, B D^{-1} B^\top, \qquad D_\gamma = D - \gamma \, B^\top A^{-1} B, \tag{7}$$

and the split then uses

$$v_\ell = w_\ell^\top A_\gamma \, w_\ell, \qquad v_r = w_r^\top D_\gamma \, w_r. \tag{8}$$

At $\gamma = 0$ this is Eq. (3) unchanged. At $\gamma = 1$ the augmented blocks are the Schur complements of D and A in Σ_v, each group's covariance *conditioned* on the other. Intermediate γ interpolates linearly in the correction term.

Both augmentations require a solve against a covariance block. Collinear assets make those blocks singular, so the implementation falls back from `np.linalg.solve` to the minimum-norm least-squares solution when the solve raises, which keeps Eq. (7) defined rather than propagating an exception out of a recursion.

Note what is *not* changed: the split rule. Equation (4) remains inverse-variance for every γ. Appendix A shows why that matters.

2.4 Hierarchical $1/N$

The third allocator ignores Σ. At level n of the tree it distributes a budget 2^{-n} equally among the leaves of each node at that level, accumulates into a running portfolio, and yields a copy. A leaf that terminates early keeps the weight it was given, so each yielded portfolio is a complete allocation over all assets and the sequence shows how the equal-weight allocation refines as the hierarchy is descended.

3 Implementation

The package is about 1,200 lines of Python over eight modules, dependent only on NumPy (Harris et al., 2020), SciPy (Virtanen et al., 2020), Polars (Vink and Polars contributors, 2025) and Plotly. Covariance and correlation frames are Polars DataFrames whose columns name the assets; the allocators translate names to indices once and then work on NumPy arrays.

Iterative traversal, not recursion. Both the SciPy-tree conversion and the allocation walk are written as explicit two-pass stack loops. This is not stylistic. Single linkage chains: on noisy data its tree depth grows with N rather than with $\log N$, and the recursive forms of these two functions raised `RecursionError` on universes that the mathematics handles without complaint. The iterative form removes the asset count as a limit.

The allocator contract. `risk_parity` and `schur_risk_parity` write *into* the tree they are given and return the same root. Every node's portfolio is replaced rather than accumulated into, which makes them idempotent: re-running on an already weighted tree — with a different covariance matrix, or a different γ — gives the same answer as running on a fresh one. The cost is that the caller's tree is modified. `Dendrogram` is a frozen dataclass, but the `Cluster` it holds is not, so passing `dendrogram.root` to an allocator rewrites the portfolios inside that dendrogram; a caller who needs the unweighted tree must deep-copy it. `one_over_n` differs on both counts: it accumulates into a local buffer, leaving the input untouched, and yields rather than returns.

Numerical conventions. Returns are simple returns from prices, with leading all-null rows dropped and remaining nulls and NaNs filled with zero. Covariance and correlation are the plain sample estimators (`np.cov`, `np.corrcoef`). No shrinkage is applied anywhere: the package's robustness claim rests on avoiding the inversion, not on regularising the estimate.

4 Numerical results

All numbers below come from the twenty-asset daily price panel shipped in the test suite as `tests/resources/stock_prices.csv` (320 observations, 20 assets). Variances are of daily returns; annualised volatility multiplies by $\sqrt{252}$. We report the effective number of positions $N_{\text{eff}} = 1/\sum_i w_i^2$ as a concentration measure — it equals N for equal weights and 1 for a single position.

Table 1: HRP allocations on the twenty-asset panel, by linkage method and tree construction. *Bisect* rebuilds the tree by halving the linkage leaf order.

Linkage	Bisect	Variance	Ann. vol.	N_{eff}	$\max_i w_i$
`single`	no	6.388×10^{-5}	0.1269	7.82	0.2245
`single`	yes	5.325×10^{-5}	0.1158	13.63	0.1308
`complete`	no	5.255×10^{-5}	0.1151	11.82	0.1591
`complete`	yes	5.131×10^{-5}	0.1137	13.00	0.1288
`average`	no	6.047×10^{-5}	0.1234	10.21	0.2042
`average`	yes	5.702×10^{-5}	0.1199	15.39	0.1003
`ward`	no	5.478×10^{-5}	0.1175	13.27	0.1377
`ward`	yes	5.397×10^{-5}	0.1166	14.05	0.1238

Table 2: Schur complementary allocation on the same panel, `ward` linkage, as γ varies. The last row is the global minimum-variance portfolio, which allows short positions, for reference.

	Variance	Ann. vol.	N_{eff}	$\max_i w_i$
$\gamma = 0.00$	5.478×10^{-5}	0.1175	13.27	0.1377
$\gamma = 0.25$	5.473×10^{-5}	0.1174	13.24	0.1381
$\gamma = 0.50$	5.467×10^{-5}	0.1174	13.20	0.1385
$\gamma = 0.75$	5.459×10^{-5}	0.1173	13.15	0.1390
$\gamma = 1.00$	5.449×10^{-5}	0.1172	13.08	0.1397
GMV (long/short)	4.137×10^{-5}	0.1021	4.37	0.2438

4.1 Tree construction

Table 1 shows the choice of linkage mattering roughly as much as one would expect from the geometry. `single` linkage is the outlier: its chaining produces the most concentrated allocation of the eight ($N_{\text{eff}} = 7.82$ against 13.27 for `ward`) and the highest variance. This is the mechanism, not an accident — an unbalanced tree splits a budget unevenly by construction, because a deep leaf has been halved more often than a shallow one.

Bisection improves every method on both measures here, and improves `single` most (variance -17%, N_{eff} from 7.82 to 13.63), which is consistent with balance being what it supplies. We would not read the uniform improvement as general: this is one panel, and the ordering of two allocators on one covariance estimate is not evidence about their ordering out of sample.

4.2 The Schur parameter

Table 2 is mostly a null result. Moving γ from 0 to 1 — from no cross-block information to the full Schur complement — reduces the portfolio variance by 0.53% and the annualised volatility by 3 basis points. The allocation barely moves: the largest weight changes by 0.002, and N_{eff} falls from 13.27 to 13.08.

The reference row explains why that is worth stating. The global minimum-variance portfolio on this covariance matrix has variance 4.137×10^{-5}, a quarter below the $\gamma = 1$ allocation, and it is a visibly different object: $N_{\text{eff}} = 4.37$, and it holds short positions (its smallest weight is -0.132). Whatever γ interpolates towards in this implementation, on this panel it is not that portfolio. Appendix A explains why, and checks the three properties the package documents for γ.

5 Constrained HRP via the critical line algorithm

Let w^{H} be the HRP portfolio computed under the risk model Σ, and let

$$\mathcal{P} = \{\, w : Aw = b,\ Gw \le g,\ l \le w \le u \,\} \tag{9}$$

be the feasible set: equality constraints such as the budget, general linear inequalities such as sector limits, and bounds on every weight. We seek

$$w^{\star} = \underset{w \in \mathcal{P}}{\arg\min}\ \tfrac{1}{2}(w - w^{\mathrm{H}})^{\top}\Sigma\,(w - w^{\mathrm{H}}), \tag{10}$$

the feasible portfolio with the smallest tracking-error variance to HRP. The distance is measured by the same risk model that produced w^{H}, which is what links the two parts of the paper.

Reduction to the CLA path. Expanding Eq. (10) gives $\frac{1}{2}w^{\top}\Sigma w - (\Sigma w^{\mathrm{H}})^{\top}w$ plus a constant. The critical line algorithm (Schmelzer and Hastie, 2026) traces, for every $\lambda \ge 0$, the solution $w(\lambda)$ of

$$\min_{w \in \mathcal{P}}\ \tfrac{1}{2}w^{\top}\Sigma w - \lambda\,\mu^{\top}w, \tag{11}$$

as a piecewise-linear path between finitely many turning points. With expected returns set to $\mu = \Sigma w^{\mathrm{H}}$, Eq. (11) at $\lambda = 1$ is Eq. (10), so $w^{\star} = w(1)$ is read off the path by linear interpolation between the two turning points that bracket $\lambda = 1$.

Properties. Three facts follow directly, assuming Σ is positive definite so that the minimiser is unique.

- If $w^{\mathrm{H}} \in \mathcal{P}$, then $w^{\star} = w^{\mathrm{H}}$: the unconstrained minimiser of Eq. (10) is feasible. Adding constraints that HRP already satisfies changes nothing.
- $w(0)$ is the minimum-variance portfolio in $\mathcal{P}$. The path from $\lambda = 0$ to $\lambda = 1$ therefore runs from constrained minimum variance to constrained HRP, and the whole family comes from a single CLA run.
- The construction is agnostic to how w^{H} was obtained. Any allocation scheme from Part I, under any risk model, can serve as the target.

5.1 Numerical results

Table 3: Constrained HRP on the twenty-asset panel, `ward` linkage. All rows also impose the budget $\mathbf{1}^{\top}w = 1$. TE is the annualised tracking error to the unconstrained HRP portfolio, in basis points. Sectors: Technology (GOOG, AAPL, FB, BABA, AMZN, AMD), Financials (BAC, MA, JPM), Consumer (WMT, UAA, SHLD, BBY, SBUX, GM), Energy (XOM, RRC), Other (GE, T, PFE).

		nearest to HRP, $w(1)$			minimum variance, $w(0)$		
Constraints	TPs	Ann. vol.	TE	N_{eff}	Ann. vol.	TE	N_{eff}
$0 \le w$	30	0.1175	0	13.27	0.1048	443	6.20
$0 \le w \le 0.10$	38	0.1182	64	14.21	0.1110	284	11.13
$0 \le w$, sectors ≤ 0.25	31	0.1171	88	13.41	0.1065	384	7.16
$0.02 \le w \le 0.10$, sectors ≤ 0.25	38	0.1202	151	14.43	0.1179	199	13.69

Table 3 applies the construction to the HRP portfolio of Table 1 (`ward`, no bisection) under four constraint sets of increasing strength. Each row is a single run of `cvxcla` (Schmelzer, 2026)

with $\mu = \Sigma w^{\mathrm{H}}$. The nearest portfolio $w(1)$ is interpolated at $\lambda = 1$, and the endpoint $w(0)$ of the same run is the minimum-variance portfolio under the same constraints. TPs counts the turning points on the path. As an independent check, both points were recomputed for every row as a quadratic program with an interior-point solver; the largest weight difference over all eight solves is 9×10^{-9}.

Feasible HRP is returned unchanged. Under long-only constraints alone the HRP portfolio is already feasible, and $w(1)$ reproduces it to 8×10^{-16}, as the first property predicts. The other endpoint of the same path, the long-only minimum-variance portfolio, has a tracking error of 443 basis points to HRP and $N_{\text{eff}} = 6.20$ against HRP's 13.27. Since HRP is the target, the path spans the whole range from a concentrated, low-variance portfolio to HRP itself.

Binding constraints cost little tracking error. HRP's largest positions exceed 10%. Capping every position at 10% binds for WMT, XOM and SBUX and costs 64 basis points of tracking error. The weight released by the capped names is spread over the remaining assets, so the cap raises N_{eff} from 13.27 to 14.21 and volatility by 7 basis points. The constrained minimum-variance portfolio is 284 basis points away from HRP, more than four times as far.

The projection moves weight along correlations, not pro rata. HRP holds 32.5% in Consumer and 24.1% in Technology. A 25% cap per sector binds for Consumer, as expected, but in the nearest portfolio it also binds for Technology, which HRP left below the cap. Measured in the Σ-norm, the cheapest way to shed Consumer weight is to move it to the assets whose returns best replace it, and on this panel some of those sit in Technology. Clipping Consumer and rescaling the rest pro rata would ignore this. Here the constraint even lowers volatility slightly, from 0.1175 to 0.1171.

Tighter constraints bring the endpoints together. With position floors of 2%, caps of 10% and the sector caps combined, eight assets sit at their floor, three at their cap, and both sector caps bind. The tracking error of $w(1)$ rises to 151 basis points, while that of the minimum-variance endpoint falls to 199. The feasible set has shrunk enough that the two ends of the path are no longer far apart.

6 Limitations

The covariance estimator is the plain sample one. On a universe whose asset count approaches its observation count this estimator is badly conditioned, and while HRP never inverts it, the block variances of Eq. (3) are still computed from it and the Schur augmentation of Eq. (7) does solve against it. Shrinkage (Ledoit and Wolf, 2004) is left to the caller, who may pass any covariance frame.

The hierarchical allocators are unconditionally long-only and fully invested; constraints enter only through the projection of Section 5, which covers linear constraints but not turnover limits or transaction costs. There is no rebalancing logic: the package maps one covariance estimate to one allocation. Nothing here evaluates HRP as a strategy. All variances reported are in-sample, computed against the same covariance matrix that produced the weights, and are properties of the construction rather than evidence about performance — the out-of-sample question that motivates HRP (López de Prado, 2016) is outside what this package measures.

7 Conclusion

HRP as published fuses a covariance estimate, a tree and a split rule into one pipeline. Part I separated them. The risk model is an input, so a shrunk or factor-model covariance replaces the sample estimate without touching the allocator. The allocation scheme is a rule applied to a tree, so HRP, the Schur split and hierarchical $1/N$ run interchangeably on the same tree, and the returned tree records the budget of every cluster. On the shipped panel the tree matters more than the rule: bisection improves balance most where the linkage tree is worst, while the Schur parameter γ moves the allocation by fractions of a percent (Appendix A).

Part II showed that HRP needs no new recursion to respect constraints. The feasible portfolio nearest to HRP in the risk model's own metric is the point $\lambda = 1$ on a critical-line path with expected returns Σw^{H}, so one CLA run handles any polyhedral constraint set and also returns the constrained minimum-variance portfolio at $\lambda = 0$. On the panel, a feasible HRP portfolio comes back unchanged. A 10% position cap costs 64 basis points of tracking error, and sector caps redistribute weight along correlations rather than pro rata.

The two parts meet in Σ. The risk model that builds the tree and sizes each split also defines what "closest to HRP" means once constraints bind, so a better risk model improves both the target and the projection. All results here are in-sample on one panel; whether constrained HRP keeps the out-of-sample behaviour that motivates HRP is the natural next question.

A The Schur allocator's documented claims

The package documents three properties of the Schur allocator. We tested each against the panel of Section 4 and against synthetic data.

$\gamma = 0$ **reproduces HRP exactly. Confirmed.** The two allocators agree to the last bit on this panel: the maximum absolute weight difference over the twenty assets is exactly 0, not merely below a tolerance. This is structural rather than fortunate — at $\gamma = 0$ Eq. (7) leaves A and D untouched, so the two code paths compute the same floating-point expression — and it makes `schur_hrp` a safe drop-in for `hrp`.

$\gamma = 1$ **recovers the minimum-variance portfolio. Not as stated.** Table 2 already shows the two portfolios differing substantially on the twenty-asset panel. The reason is visible in a case small enough to check by hand. Take

$$\Sigma = \begin{pmatrix} 4 & 1.5 \\ 1.5 & 1 \end{pmatrix}, \tag{12}$$

and the two-leaf tree. The global minimum-variance portfolio is $w^\star = (-0.25,\ 1.25)$; the Schur allocator at $\gamma = 1$ returns $(0.2,\ 0.8)$.

The mechanism is Eq. (4). Augmenting the blocks changes *which* variances are compared, but the comparison is still an inverse-variance split, and an inverse-variance split of two sub-portfolios is not the minimum-variance combination of them — the minimum-variance weight for two correlated blocks depends on their covariance, and no reweighting of the diagonal blocks alone can reproduce it. A further consequence: because both α's in Eq. (4) are non-negative, the output is long-only for every γ, while the minimum-variance portfolio generally is not. The two cannot coincide whenever $w^\star$ has a negative entry, whatever γ does.

We are describing a gap between this implementation and the sentence in its own documentation, not disputing Cotton (2024); recovering minimum variance requires the split itself to become a minimum-variance solve, which Eq. (4) never becomes.

Variance falls as γ rises. True here, not in general. It holds on the shipped panel, monotonically, as Table 2 shows. It is not a property of the algorithm. Over 200 synthetic eight-asset panels (60 observations each, independent Gaussian log-returns; see Appendix B), the variance failed to be non-increasing in γ in 76 cases — 38%. In the first such case the variance *rose* monotonically, from 1.0367×10^{-5} at $\gamma = 0$ to 1.0401×10^{-5} at $\gamma = 1$, an increase of 0.33%.

The effect is small in both directions, which is the practical point: γ is a weak knob on this construction. A user who reads "raising γ trades robustness for lower in-sample variance" should not expect the trade to be reliably available.

B Reproducibility

Every number in Section 4 was produced with `pyhrp` 2.3.3 and the panel in the repository. Tables 1 and 2:

```
import numpy as np, polars as pl
from pyhrp import hrp, schur_hrp, compute_returns, compute_cov

prices = pl.read_csv("tests/resources/stock_prices.csv",
                     try_parse_dates=True).drop("date")
cov = compute_cov(compute_returns(prices))

for method in ["single", "complete", "average", "ward"]:
    for bisection in [False, True]:
        root = hrp(prices=prices, method=method, bisection=bisection)
        w = np.array(list(root.portfolio.weights.values()))
        print(method, bisection, root.portfolio.variance(cov),
              1 / np.sum(w ** 2), w.max())

for gamma in [0.0, 0.25, 0.5, 0.75, 1.0]:
    root = schur_hrp(prices=prices, method="ward", gamma=gamma)
    print(gamma, root.portfolio.variance(cov))
```

The minimum-variance reference row is $w^\star \propto \Sigma^{-1}\mathbf{1}$, normalised to sum to one. The monotonicity experiment of Appendix A:

```
rng = np.random.default_rng(0)
violations = 0
for _ in range(200):
    X = rng.standard_normal((60, 8))
    p = pl.DataFrame({f"A{i}": 100 * np.exp(np.cumsum(X[:, i] * 0.01))
                      for i in range(8)})
    c = compute_cov(compute_returns(p))
    v = [schur_hrp(prices=p, method="ward", gamma=g).portfolio.variance(c)
         for g in (0.0, 0.5, 1.0)]
    violations += not (v[0] >= v[1] >= v[2] - 1e-18)
print(violations, "/ 200")
```

The constrained results of Table 3 use `cvxcla` 2.3.4 (Schmelzer, 2026). The path is affine in λ between turning points, so $w(1)$ is interpolated exactly from the two that bracket $\lambda = 1$:

```
from cvxcla import CLA

S = cov.to_numpy()
names = cov.columns
root = hrp(prices=prices, method="ward")
wh = np.array([root.portfolio.weights[a] for a in names])
n = len(names)

cla = CLA(mean=S @ wh, covariance=S,
          lower_bounds=np.full(n, 0.0), upper_bounds=np.full(n, 0.10),
          a=np.ones((1, n)), b=np.ones(1),
          g=G_sector, h=np.full(5, 0.25))   # optional sector rows
tps = cla.turning_points                    # lambda decreasing to 0
hi, lo = next((p, q) for p, q in zip(tps, tps[1:])
              if p.lamb >= 1.0 >= q.lamb)
t = (hi.lamb - 1.0) / (hi.lamb - lo.lamb)
w_star = (1 - t) * hi.weights + t * lo.weights   # nearest to HRP
w_minvar = tps[-1].weights                       # lambda = 0
```

References

Peter Cotton. Schur complementary allocation: A unification of hierarchical risk parity and minimum variance portfolios. *arXiv preprint arXiv:2411.05807*, 2024. URL `https://arxiv.org/abs/2411.05807`.

Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with numpy. *Nature*, 585:357–362, 2020.

Olivier Ledoit and Michael Wolf. A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariate Analysis*, 88(2):365–411, 2004.

Marcos López de Prado. Building diversified portfolios that outperform out of sample. *The Journal of Portfolio Management*, 42(4):59–69, 2016. doi: 10.3905/jpm.2016.42.4.059.

Harry Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.

Harry M. Markowitz. The optimization of a quadratic function subject to linear constraints. *Naval Research Logistics Quarterly*, 3(1–2):111–133, 1956.

Richard O. Michaud. The markowitz optimization enigma: Is 'optimized' optimal? *Financial Analysts Journal*, 45(1):31–42, 1989.

Thomas Schmelzer. cvxcla: Critical line algorithm for portfolio optimization. `https://github.com/cvxgrp/cvxcla`, 2026.

Thomas Schmelzer and Trevor Hastie. The critical line algorithm and the constrained LASSO: One curve, two literatures. *arXiv preprint arXiv:2609.25704*, 2026. URL `https://arxiv.org/abs/2609.25704`.

Ritchie Vink and Polars contributors. Polars: Dataframes for the new era. `https://pola.rs`, 2025.

Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. Scipy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17:261–272, 2020.

Joe H. Ward. Hierarchical grouping to optimize an objective function. *Journal of the American Statistical Association*, 58(301):236–244, 1963.